Continuous Build and Delivery Assignment 2025/26 – Weighting 30%

1. Assignment Outline

The objective of this assignment is to take one microservice (for example, one developed in the MicroServices Architecture module) and demonstrate its journey through a **complete Continuous Integration/Continuous Deployment (CI/CD) pipeline**. The emphasis of this assignment is on:

- Automation from code commit to deployment
- Clear alignment with CI/CD and DEVOPS principles
- Evidence-based demonstration of testing, code quality, and deployment.

You can use your local machine, cloud infrastructure (e.g. AWS) or a combination of both. Tool sophistication is not a grading criterion; clarity, automation, and rationale are.

2. What to include in the Pipeline

Version Control

- Use version control on the developer machine
- Push code to a remote repository. (Bitbucket or Github)

Build

- Automatically clone from the repository and build the application using Maven (or equivalent. build tool).
- Package the application into a deployable artefact (e.g. a.jar or .war)

Code Analysis

- Integrate automated static code analysis (e.g. SonarQube – recommended, PMD or Checkstyle)
- Define and enforce **quality gates** (automated pass/fail criteria such as minimum coverage or no blocker issues)

Testing

- Develop a **clear test strategy aligned with the Test Pyramid**
- Include multiple test levels where appropriate (unit, integration, and optionally end-to-end)
- Tests much be automatically triggered by the CI server.

Deployment

- Create a Docker image for the application.
- Automate deployment using a tool such as Ansible or scripted automation.
- Deploy to a local server or cloud environment

Automation and Failure Handling

- The pipeline should fail automatically when a stage fails (e.g. build, test or quality gate)

3. Submission Requirements

To be eligible for grading, all of the following must be submitted:

A **ZIP file** containing:

- Source code
- Report (Word or PDF)
- Screencast recording
- A **link to the Github (or Bitbucket) repository** that is publicly accessible or accessible to the lecturer.

The demonstrated pipeline execution **must correspond to the same repository and commit history.** The commit shown in the screencast **must be identifiable by commit message.**

Failure to meet these requirements may result in the submission being marked incomplete.

Report Content

The report should include:

1. **Introduction to CI/CD**
   - Explain Continuous Integration vs. Continuous Delivery
   - Discuss current state-of-the-art practices and tools
   - Reference all external sources using **Harvard referencing**
2. **User Stories**
   - List the User Stories for the service using Who/What/Why format
   - Include acceptance criteria

3. **High-Level Architecture**
    - Describe the application architecture
    - Use diagrams
4. **Test Strategy**
    - Describe your testing approach
    - Explain how it aligns with the Test Pyramid
5. **Pipeline Description**
    - Describe each pipeline stage (2-3 sentences per stage)
    - List key configuration points or commends per stage
    - Justify tool choices
    - Include screenshots or diagrams as evidence
6. **Evaluation and Reflection**
    - Evaluate pipeline execution time and automation level
    - Identify at least one **conscious trade-off** made (e.g. speed vs coverage)
    - Suggest concrete improvements or alternative tools
7. **Repository**
    - **Link to Github (or Bitbucket)** repository that is accessible.

Screencast Requirements (Maximum 10 minutes with camera on)

The screencast is a structured walkthrough, not an unedited live demo. Clear narration and visible evidence are essential.

The screencast must include:

1. **Context and Rationale**
    - Overview of the pipeline stages
    - Justification for major tool choices
2. **Pipeline Trigger**
    - Demonstrate a small code change
    - Show automatic pipeline execution triggered by a code push
3. **Build and Test Execution**
    - Show build stage (artefact created)
    - Show test execution and results
    - Explicitly reference the Test Pyramid

4. **Code Quality Analysis**
   - Show static analysis results and quality gate status
5. **Deployment**
   - Show Docker image creation and deployment
   - Demonstrate a running application or container
6. **Evaluation**
   - Comment on pipeline speed and automation level
   - Briefly describe one limitation and one improvement

4. Marking Rubric

1. Report (40%)

1.1 Introduction to CI/CD (5%)

Level	Criteria
Excellent (70%+)	Clear and accurate explanation of Continuous Integration vs. Continuous Delivery. Insightful discussion of modern CI/CD practices and tools. All sources correctly referenced using Harvard style.
Good (55–69%)	Correct explanation of CI/CD with some discussion of modern tools and practices. Minor issues with depth or referencing.
Satisfactory (40–54%)	Basic explanation with limited discussion of current practices. Referencing present but inconsistent or shallow.
Fail (0–39%)	Inaccurate or missing explanation. No meaningful discussion of modern CI/CD. Poor or missing references.

1.2 User Stories (5%)

Level	Criteria
Excellent	User stories clearly written in Who/What/Why format with well-defined acceptance criteria. Stories are relevant and complete.
Good	User stories mostly well-structured with acceptance criteria. Minor gaps in clarity or completeness.
Satisfactory	Basic user stories provided, limited acceptance criteria or inconsistent formatting.
Fail	User stories missing, poorly structured, or not aligned with the service.

1.3 High-Level Architecture (5%)

Level	Criteria
Excellent	Clear architectural description supported by well-labelled diagrams. Architecture choices are easy to understand.
Good	Architecture described with diagrams, but some components lack clarity or explanation.
Satisfactory	Basic architectural description with minimal or unclear diagrams.
Fail	Architecture missing or incomprehensible. No diagrams.

1.4 Test Strategy (5%)

Level	Criteria
Excellent	Comprehensive testing strategy aligned with the Test Pyramid. Clear explanation of test levels (unit, integration, optional end-to-end) and tooling.
Good	Good test coverage but partial alignment with the Test Pyramid. Some test levels missing or not explained.
Satisfactory	Limited testing strategy (e.g. unit tests only). Weak alignment with Test Pyramid.
Fail	No meaningful test strategy or automation described.

1.5 Pipeline Description (10%)

Level	Criteria
Excellent	Each pipeline stage clearly described (2–3 sentences per stage). Key configuration points or commands included. Tool choices justified. Screenshots or diagrams provided as evidence.
Good	Pipeline stages described but some lack detail or justification. Limited supporting evidence.
Satisfactory	Basic pipeline description with minimal technical detail and little justification.
Fail	Pipeline stages unclear, missing, or incorrectly described.

1.6 Evaluation and Reflection (10%)

Level	Criteria
Excellent	Clear evaluation of pipeline execution time and automation level. At least one conscious trade-off identified. Concrete improvements or alternatives suggested.
Good	Some evaluation provided with limited depth. Trade-offs or improvements mentioned but not well developed.
Satisfactory	Minimal evaluation. Observations are superficial or generic.
Fail	No evaluation or reflection provided.

1.7 Repository Link (Pass/Fail – Required)

- GitHub (or Bitbucket) repository link provided
- Repository is accessible
- Repository content matches report and screencast

2. Screencast (60%)

Maximum length: 10 minutes – Camera on

The screencast must be a **structured walkthrough**, not an unedited live demo. All claims must be supported by **on-screen evidence**.

2.1 Organisation and Presentation (10%)

Level	Criteria
Excellent	Logical, well-paced, professional presentation with clear narration and structure.
Good	Mostly clear and structured, minor pacing or clarity issues.
Satisfactory	Some structure but inconsistent narration or flow.
Fail	Disorganised, difficult to follow, or poorly narrated.

2.2 Context and Rationale (10%)

Level	Criteria
Excellent	Clear overview of pipeline stages. Strong justification for major tool choices.
Good	Pipeline overview provided with some justification.
Satisfactory	Basic overview with little rationale.
Fail	No meaningful context or justification.

2.3 Pipeline Trigger (10%)

Level	Criteria
Excellent	Demonstrates a small code change and shows automatic pipeline execution triggered by a code push. Commit is identifiable by message.
Good	Pipeline trigger shown but code change or commit identification is unclear.
Satisfactory	Trigger described but not clearly demonstrated.
Fail	No automated trigger demonstrated.

2.4 Build and Test Execution (15%)

Level	Criteria
Excellent	Build stage shown with artefact creation. Automated test execution and results visible. Explicit reference to the Test Pyramid.
Good	Build and tests shown but limited discussion of test strategy.
Satisfactory	Build shown with minimal testing evidence.
Fail	No working build or test execution demonstrated.

2.5 Code Quality Analysis (5%)

Level	Criteria
Excellent	Static analysis results shown with clear quality gate status.
Good	Code analysis shown but quality gates unclear or not enforced.
Satisfactory	Minimal or partial analysis shown.
Fail	No code quality analysis demonstrated.

2.6 Deployment (5%)

Level	Criteria
Excellent	Docker image creation and automated deployment shown. Running application or container demonstrated.
Good	Deployment shown but partially manual or limited evidence.
Satisfactory	Basic deployment with minimal automation.
Fail	No deployment demonstrated.

2.7 Evaluation (5%)

Level	Criteria
Excellent	Clear commentary on pipeline speed and automation level. One limitation and one improvement clearly stated.
Good	Evaluation provided but lacks depth or specificity.
Satisfactory	Minimal evaluation.
Fail	No evaluation provided.